KubeBalancer

An Image Classification Pipeline on Kubernetes — Milestone 1: Architecture

Course: CSC 478 — Cloud Engineering
Instructor: Dr. Linh Ngo
Student: Angeudy Brisita Perez (solo project)
Milestone: 1 of 3 — Architecture (design only)
Submitted: October 5, 2026
Repository: https://github.com/angeudybp/csc478-project
Project idea: Instructor-provided template (Image Classification Pipeline on Kubernetes)

This is a design document. No code or Kubernetes manifests are delivered at this milestone; they arrive in Milestones 2 and 3.

1. Overview

Running an image classifier on a laptop is easy. Running it as a service is harder: what happens when many people upload at once, when a request arrives before the model has loaded, when someone uploads a huge photo or a text file renamed to `.jpg`, or when the pod holding all the images is killed? These are orchestration problems, not machine-learning problems, and they are what this project is about.

KubeBalancer is a small image classification service on Kubernetes. A user uploads an image to one public endpoint and gets back a label and a confidence score. Four services split the work:

- **gateway** — validates uploads and coordinates the other services;
- **preprocess** — resizes and normalizes images for the model;
- **inference** — runs the MobileNetV2 model (scaled to 3 replicas, with a metrics sidecar);
- **storage** — saves the original image, the processed image, and the prediction so they can be retrieved later.

Everything runs on a single-node Minikube cluster, so every result in later milestones can be reproduced on a laptop. Three decisions shape the design:

1. **A central gateway** instead of services calling each other in a chain. This adds one network hop but gives the system exactly one public entrypoint and keeps the other services simple.
2. **Synchronous requests** instead of a message queue. A queue handles load better, but would shift the project's effort from Kubernetes into distributed messaging (§3.6).
3. **One service owns all stored data.** Only `storage` has a persistent volume, so no other component has to worry about saving state.

Note: all latency numbers in this document are design targets, not measurements. Real measurements first appear in Milestone 3.

2. User Stories and Data Flow

US-1 — Classify an image. *As a user, I want to upload an image and receive a predicted label with a confidence score, so that I can classify images without installing anything.*

- `POST /api/v1/images` with a JPEG or PNG under 8 MB returns 200 with the `imageId`, `label`, `confidence`, and the top 3 classes.
- Files over 8 MB are rejected with 413; files that are not images are rejected with 415. Neither crashes a pod.
- *Tests:* T1–T6.

US-2 — Retrieve a past result. *As a user, I want to retrieve a previous prediction and its images by ID, so that results are saved and can be checked later.*

- `GET /api/v1/results/{imageId}` returns the saved prediction; an unknown ID returns 404.
- Both the original and processed images can be downloaded by ID.
- Results are still available after the storage pod is deleted and recreated. *Tests:* T7, T11.

US-3 — Scale and survive failures. *As the operator, I want to scale the inference tier and lose individual pods without dropping traffic, so that the service stays available.*

- Inference scales from 1 to 3 replicas, and the metrics sidecar shows requests spread across all three.
- A deleted inference pod is recreated automatically while the others keep serving.
- If inference is scaled to zero, the gateway returns a clean 503 instead of hanging.
- `kubectl delete -f . && kubectl apply -f .` brings the whole stack back with no manual steps. *Tests:* T8–T10, T12, T13.

Figure 1 shows the data flow. An upload passes through preprocessing, inference, and storage before the response returns. Retrieval only touches the gateway and storage.

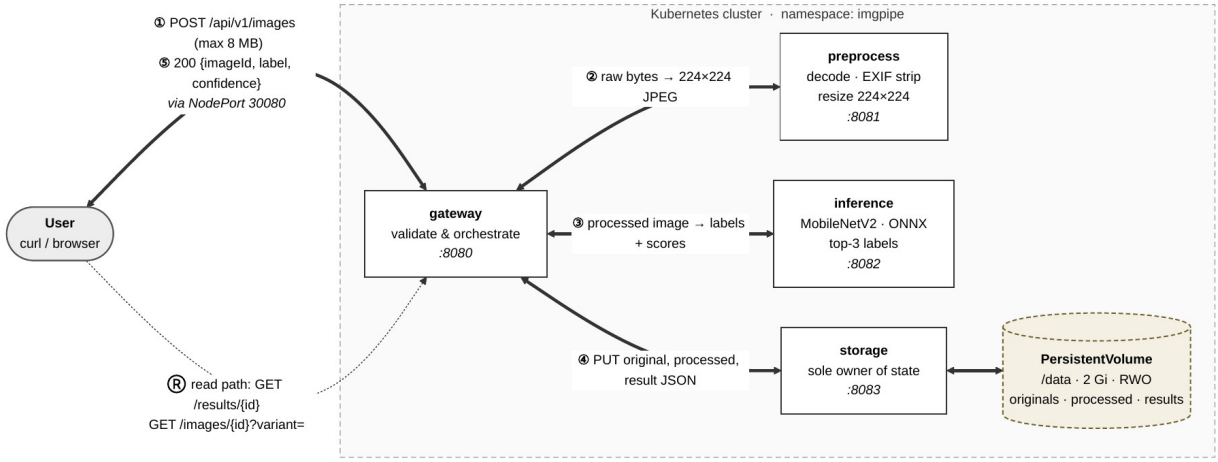


Figure 1. Data flow. Solid arrows are the upload path; dashed arrows are the retrieval path. Only the gateway is reachable from outside the cluster.

3. Architecture

The design follows the brief's constraints: exactly one external entrypoint, at least one multi-container pod, and one component scaled to three or more replicas. It targets single-node Minikube, which has no cloud load balancer and only one node for storage.

3.1 Components

Component	Containers	Replicas	Port	Responsibility
`gateway`	1	1	8080	Only public entrypoint. Validates uploads, calls the other services in order, and serves retrieval requests.
`preprocess`	1	2	8081	Decodes the image, removes metadata, resizes to 224×224, and normalizes it (Pillow).
`inference`	**2**	**3**	8082, 9100	Runs MobileNetV2 with ONNX Runtime and returns the top labels. Includes the metrics sidecar.
`storage`	1	1	8083	Owns the only persistent volume. Saves images and prediction records.

Table 1. Components. Inference is the only multi-container pod.

Why a separate gateway? Without it, the preprocessor would have to coordinate the whole pipeline, and storage would need to be public for retrieval to work. A gateway keeps preprocessing and inference focused on one job each and keeps stored data off the public network.

Why is preprocessing its own service? It is the stage most likely to change (image size, model type), and it is CPU-heavy while the gateway mostly waits on network calls. Splitting them lets each be scaled or replaced on its own.

3.2 Multi-Container Pod: Inference + Metrics Sidecar

Each inference pod runs two containers that share an `emptyDir` volume at `/var/log/app/`:

- **inference** — the model server. It writes one JSON line per request (time, request ID, latency, result) to `requests.jsonl`.
- **metrics-exporter** — the sidecar. It reads that file and serves request count, error count, and p50/p95 latency on port 9100.

They belong in one pod because the sidecar needs to read a file the app writes and must report on *that specific replica*. Separate Deployments cannot share an `emptyDir`. Putting the sidecar on the scaled component also proves load balancing: with three replicas, each one's metrics endpoint should show a similar request count. *Limitation:* metrics are kept in memory and reset on restart, and no Prometheus server is included; metrics are read with `kubectl port-forward` and `curl`.

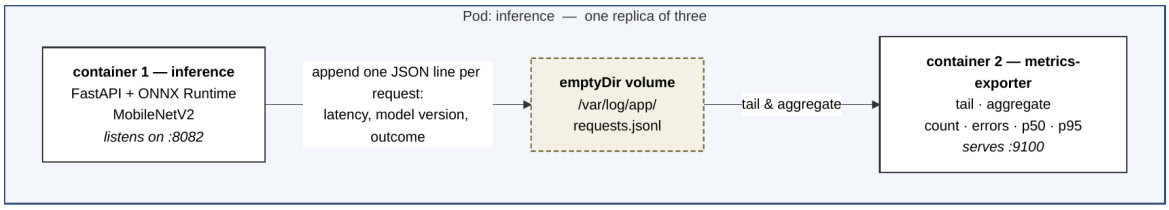


Figure 2. The inference pod. Both containers share a volume and network, so they run in one pod. Each replica has its own exporter.

3.3 Services and External Access

Service	Type	Ports	Reason
`gateway-svc`	**NodePort**	30080 → 8080	The single public entrypoint. Minikube has no cloud load balancer, so a `LoadBalancer` Service would stay `Pending`; NodePort works out of the box.
`preprocess-svc`	ClusterIP	8081	Internal only; balances across the two replicas.
`inference-svc`	ClusterIP	8082, 9100	Internal only; balances across the three replicas.
`storage-svc`	ClusterIP	8083	Internal only; only the gateway calls it.

Table 2. Service choices. Only `gateway-svc` is reachable from outside the cluster.

Services are reached by DNS name (e.g. `preprocess-svc.imgpipe.svc.cluster.local`), so no configuration changes when pods move. Figure 3 shows the full layout.

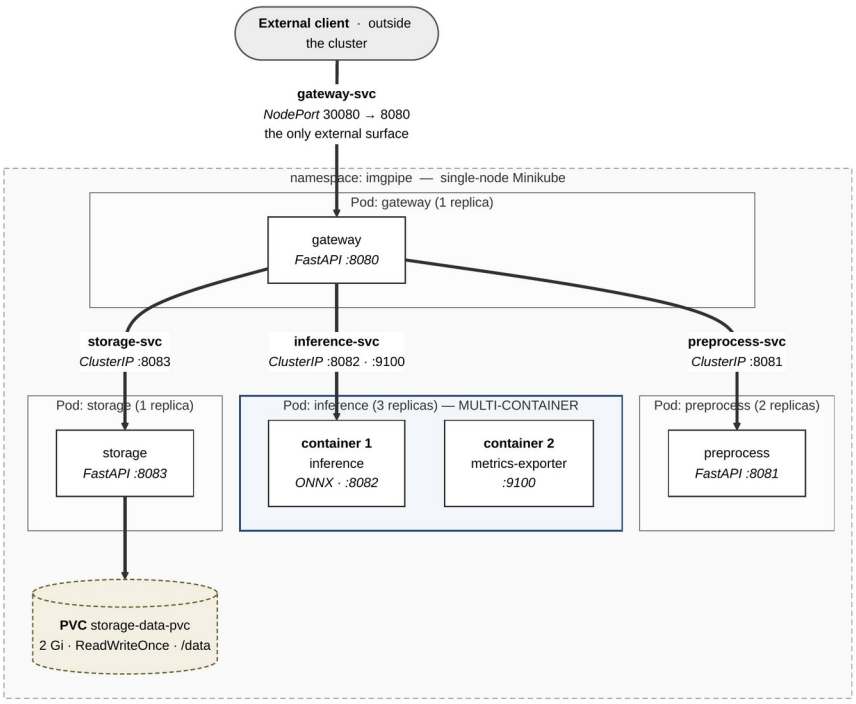


Figure 3. Kubernetes topology. Outlined regions are pods with replica counts; only `gateway-svc` crosses the cluster edge.

3.4 Request Flow and Timeouts

The gateway assigns each request an `X-Request-Id` and passes it to every service, so one request can be traced through all logs. The steps are:

1. **Validate** — check file type and the 8 MB size limit before reading the whole upload.
2. **Preprocess** — send the image to `preprocess` and get back a 224×224 JPEG.
3. **Infer** — send the processed image to `inference` and get back the top labels.
4. **Store** — save the original, processed image, and prediction to `storage`, then respond.

Hop	Timeout (target)	Retry
Client → gateway	30 s	None — the client decides.
Gateway → preprocess	5 s	Once, only if the connection failed.
Gateway → inference	15 s	**Never.** Retrying a slow, overloaded tier makes it worse.
Gateway → storage	5 s	Once, only if the connection failed.

Table 3. Timeout budget. Internal timeouts add up to less than 30 s, so a slow service produces a clean error instead of a dropped connection. These are targets, not measurements.

3.5 Scaling Target

Inference is scaled to 3 replicas. It holds no state, so replicas are interchangeable, and running the model is the most CPU-heavy step in the pipeline, so it will be the first to slow down under load. The gateway stays at 1 replica (it mostly waits on network calls), preprocess runs 2 for availability, and storage is fixed at 1 because its volume can only be attached to one pod (§5).

3.6 Main Tradeoff: Synchronous Requests

The gateway keeps the client connection open for the whole pipeline and returns the prediction in the same response. The alternative is a queue: accept the upload, return a job ID, and let workers process it in the background. A queue handles traffic spikes better, but it adds a message broker, retry handling, and job tracking — a lot of work that is

not about Kubernetes. The synchronous design is simpler, and its failures are easy to see. The cost: **when inference is overloaded, requests time out instead of waiting in line.** Adding a queue is planned as a future improvement.

3.7 Reliability Settings

- **Health checks.** Every service has `/healthz` (is the process alive?) and `/readyz` (can it handle traffic?). Inference only reports ready after the model has loaded, so new pods never receive requests too early.
- **Resource limits.** Inference requests 256 Mi memory / 0.25 CPU, limited to 768 Mi / 1 CPU, with `OMP_NUM_THREADS=1` so three replicas do not fight over CPU. Other services: 64 Mi / 0.05 CPU, limited to 256 Mi / 0.5 CPU.
- **Images.** Fixed version tags (never `latest`), loaded into Minikube with `minikube image load`. Model weights are built into the image so pods start without downloading anything.

3.8 Repository Layout

Everything runs in the `imgpipe` namespace. Manifest files are numbered so `kubectl apply -f .` creates them in the right order:

```
deployments/   00-namespace.yaml  10-gateway.yaml  11-preprocess.yaml
               12-inference.yaml  13-storage.yaml
services/      20-gateway-svc.yaml  21-preprocess-svc.yaml
               22-inference-svc.yaml  23-storage-svc.yaml
storage/       05-storage-pvc.yaml
services-src/  gateway/  preprocess/  inference/  metrics-sidecar/  storage/
```

4. API Contracts

All services use JSON over HTTP (FastAPI, Python 3.11). Every error uses the same format so the gateway can pass errors along consistently:

```
{ "error": { "code": "PAYLOAD_TOO_LARGE", "message": "Upload exceeds 8 MB" },
  "requestId": "7f3c1e9a" }
```

4.1 Public API (gateway)

Endpoint	Purpose
`POST /api/v1/images`	Upload an image (multipart, JPEG/PNG, max 8 MB) and get a prediction.
`GET /api/v1/results/{imageId}`	Get a saved prediction.
`GET /api/v1/images/{imageId}?variant=original|processed`	Download the original or processed image.
`GET /api/v1/results?limit=20`	List recent predictions, newest first.

Table 4. Gateway endpoints.

Example request and response:

```
curl -X POST "http://$(minikube ip):30080/api/v1/images" -F "file=@dog.jpg"

{ "imageId": "img_01JAR7K2M9QX4T8B",
  "label": "golden retriever", "confidence": 0.8734,
  "topK": [ {"label": "golden retriever", "score": 0.8734},
            {"label": "Labrador retriever", "score": 0.0612},
            {"label": "kuvasz", "score": 0.0188} ],
  "modelVersion": "mobilenetv2-1.0-onnx", "mock": false,
  "timings": {"preprocessMs": 38, "inferenceMs": 121, "storageMs": 44, "totalMs": 207},
  "requestId": "7f3c1e9a" }
```

Timings above are example values only.

Code	Error code	When
200	—	Success.
400	`EMPTY_UPLOAD`	Empty file or missing `file` field.
404	`NOT_FOUND`	Unknown `imageId`.
413	`PAYLOAD_TOO_LARGE`	Upload over 8 MB.
415	`UNSUPPORTED_MEDIA_TYPE`	Not a JPEG/PNG, or not a valid image.
502	`UPSTREAM_FAILED`	An internal service returned an error.
503	`UPSTREAM_UNAVAILABLE`	No healthy replica of an internal service.
504	`UPSTREAM_TIMEOUT`	An internal service took too long (Table 3).

Table 5. Gateway status codes.

4.2 Internal APIs

Service	Endpoint	Input → Output
preprocess	`POST /preprocess`	Raw image bytes → 224×224 JPEG (base64) with original dimensions. 415 if not an image.
inference	`POST /predict`	`{ data, topK }` → top labels and scores, model version, `mock` flag.
inference sidecar	`GET /metrics` (port 9100)	Request count, error count, p50/p95 latency (Prometheus format).
storage	`PUT` / `GET /objects/{imageId}/{variant}`	Save or fetch image bytes (`original` or `processed`).
storage	`PUT` / `GET /results/{imageId}`	Save or fetch a prediction record. Saving the same ID twice overwrites it.
storage	`GET /results?limit=n`	List recent records.
all	`GET /healthz`, `GET /readyz`	Liveness and readiness checks (§3.7).

Table 6. Internal endpoints. None are reachable from outside the cluster.

Mock mode. With `MOCK_MODE=1`, inference returns a predictable fake label based on the image bytes, so the pipeline can be built and tested without the real model. Every response includes a `mock` field so fake results are never mistaken for real ones.

5. Persistence Plan

Milestone 2 uses temporary storage; Milestone 3 adds the persistent volume described here. Each upload produces three saved items:

Item	Size (est.)	Why it is saved
Original image	0.1–8 MB	Lets a prediction be re-run later without re-uploading.
Processed image (224×224)	~20 KB	Shows exactly what the model saw.
Prediction record (JSON)	~1 KB	The result: labels, scores, model version, timings, timestamp.

Table 7. Data saved per upload. Logs are not saved to disk; they are read with `kubectl logs`.

Volume. One PersistentVolumeClaim, `storage-data-pvc`: 2 Gi, ReadWriteOnce, Minikube's default `standard` storage class, mounted at `/data` in the storage pod only. Reclaim policy is `Retain`, so deleting the claim does not immediately delete the data. A manually created `hostPath` volume is the backup plan if automatic provisioning fails.

```
/data/
├── originals/<yyyy>/<mm>/<dd>/<imageId>.<ext>   original upload
├── processed/<yyyy>/<mm>/<dd>/<imageId>.jpg     224x224 model input
├── results/<imageId>.json                       prediction record
└── index/manifest.jsonl                         one line per upload (for listing)
```

Images are grouped by date so no folder grows too large. The index file is append-only, which makes listing recent results fast and avoids rewriting a shared file.

Single-writer limit. A ReadWriteOnce volume can only be attached to one pod, so storage runs **1 replica** with the `Recreate` update strategy (a rolling update would get stuck trying to attach the volume to a second pod). The tradeoff is a short write outage during storage restarts; uploads in that window get a 503. In return, there is never more than one process writing files, so no locking is needed. Moving to an object store such as MinIO would remove this limit and is a possible next step.

Verification (Milestone 3):

1. Upload three images and record their IDs.
2. Confirm all three can be retrieved.
3. Delete the storage pod: `kubectl delete pod -l app.kubernetes.io/component=storage -n imgpipe`.
4. Wait for Kubernetes to recreate it, then retrieve all three again — the results must match exactly.
5. Repeat with `kubectl rollout restart deployment/storage` and with a full `kubectl delete -f . && kubectl apply -f .`, recording whether data survives each.

6. Risk and Test Plan

#	Risk	L / I	Mitigation	Test
R1	**Large model image** slows pod startup or fails to load on Minikube.	M / M	Small model (MobileNetV2, ~14 MB) on a slim base image; weights built in; `imagePullPolicy: IfNotPresent`.	T12
R2	**Timeouts cascade**: one slow service hangs the whole request.	M / H	Per-hop timeouts that add up to less than the client timeout; no retries on errors.	T8
R3	**Out of memory** with three inference replicas on one node.	M / H	Memory/CPU limits and `OMP_NUM_THREADS=1`.	T10, T13
R4	**Storage volume** cannot attach to two pods.	L / H	Storage fixed at 1 replica with `Recreate`.	T11
R5	**Requests reach a pod before the model loads.**	H / M	Separate readiness check that waits for the model.	T9, T14
R6	**Huge or malicious images** exhaust memory.	M / H	8 MB limit checked early, file-type allowlist, pixel-count limit in Pillow.	T2, T4, T5
R7	**Teardown/rebuild fails** for grading.	M / H	Numbered manifests, dedicated namespace, fixed NodePort.	T12
R8	**Solo schedule**: getting stuck on the model blocks everything.	M / M	Mock mode lets the pipeline be built without the real model.	T1

Table 8. Top risks. L = likelihood, I = impact (Low / Medium / High).

Test approach. Unit tests (`pytest`) cover each service's logic. Each service is then tested alone with `curl`. Integration tests run the full pipeline through the public endpoint using a script in the repository. Operational tests (scaling, pod deletion, persistence) use `kubectl`, and their output is saved as evidence. Milestone 2 uses mock mode so that Kubernetes problems are not confused with model problems.

ID	Case	Action	Expected result	When
T1	Happy path	Upload a 500 KB dog JPEG	200 with a label; three items saved	M2
T2	Too large	Upload a 12 MB JPEG	413; no pod restart	M2
T3	Near limit	Upload a 7.5 MB JPEG	200 within 30 s	M2
T4	Invalid image	Text file renamed `.jpg`	415; no crash	M2
T5	Wrong format	Upload GIF and SVG	415 for both	M2
T6	Empty file	Upload a 0-byte file	400	M2
T7	Unknown ID	`GET /results/img_doesnotexist`	404	M2
T8	Service down	Scale inference to 0	Quick 503, recovers when scaled back	M2
T9	Self-healing	Delete one inference pod	Pod recreated; other requests keep working	M2
T10	Scaling	Scale inference to 3	All 3 ready; sidecars show balanced requests	M2
T11	Persistence	Upload 3, delete storage pod, re-fetch	Identical results after restart	M3
T12	Clean rebuild	`kubectl delete -f . && kubectl apply -f .`	Everything ready, no manual steps	M2
T13	Concurrency	10 uploads at once	All 200; first measured p95 latency	M2
T14	Readiness	Restart inference, poll `/readyz`	Not ready until model loads; no early traffic	M2

Table 9. Test matrix. M3 tests need the persistent volume.

7. Team Charter

This is a solo project, so the charter defines roles, a working schedule, and a Definition of Done for one person.

Role	Responsibility	Effort
Platform	Minikube, manifests, Services, health checks, resource limits, PV/PVC, teardown and rebuild.	~45%
Services	The four FastAPI apps, the metrics sidecar, Dockerfiles, the model and mock mode.	~35%
Docs & demo	Reports, diagrams, evidence screenshots, demo video.	~20%

Table 10. Roles. Platform gets the most time because the project is graded on Kubernetes work.

Checkpoint	How often	Output
Work session	2–3 per week, ~2 hours	Commits pushed; nothing left uncommitted.
Self-review	Weekly	Short note in `docs/journal.md`: progress, updated risks, blockers.
Milestone check	Before each submission	Go through every deliverable on the checklist.

Table 11. Working schedule.

Definition of Done. A task is done only when all of these are true:

1. It deploys on a fresh cluster with no manual steps.
2. It works when tested with a `curl` command saved in the repository.
3. Its test from Table 9 passes, and the output is saved.
4. `kubectl delete -f . && kubectl apply -f .` still works.
5. Any change from this design is written down with the reason.
6. It is committed with a clear message.

Milestone	Scope
M1	Architecture and design (this document).
M2	Deployments, Services, sidecar, scaling and self-healing evidence; no persistence.
M3	PV/PVC, persistence checks, health checks, resource limits, demo video, final report.

Table 12. Milestones. In M2, the mock pipeline is built first so a working end-to-end system exists early.